

Execute this reproducible Python pipeline sequentially to transform the raw operational CSVs into the validated Golden Dataset.

Entity Resolution (Accounts & Borrowers)

Eliminate ghost accounts and consolidate duplicate human profiles to prevent baseline inflation.

Entity Resolution Pipeline

Python

```
import pandas as pd

# 1. Purge ghost accounts missing a valid borrower entity
df_accounts_clean = df_accounts.dropna(subset=['borrower_id'])

# 2. Chronological sorting to keep the oldest borrower profile as the master
df_borrowers = df_borrowers.sort_values('created_at')

# 3. Create master ID mapping for duplicates (matching name and email)
df_borrowers['master_borrower_id'] = df_borrowers.groupby(['name', 'email'])['borrower_id'].transform('first')
borrower_id_mapping = dict(zip(df_borrowers['borrower_id'], df_borrowers['master_borrower_id']))

# 4. Drop redundant duplicate borrower profiles
df_borrowers_clean = df_borrowers.drop_duplicates(subset=['name', 'email'], keep='first')

# 5. Map the master IDs back to the accounts table and merge
df_accounts_clean['borrower_id'] = df_accounts_clean['borrower_id'].map(borrower_id_mapping).fillna(df_accounts_clean['borrower_id'])
df_master_entity = pd.merge(df_accounts_clean, df_borrowers_clean, left_on='borrower_id', right_on='master_borrower_id', how='left').drop(columns=['master_borrower_id'])
```

Temporal Standardization & Unified Touchpoint Ledger

Convert all disparate interaction logs to UTC and stack them into a single chronological timeline.

Standardization and Ledger Construction

Python

```
# 1. Standardize timestamp columns to UTC across operational
dataframes
timestamp_cols = ['event_at', 'interaction_time', 'created_at',
'updated_at']
operational_dfs = [df_calls, df_whatsapp_events, df_sms_events,
df_field_visits, df_master_entity, df_clean_payments]

for df in operational_dfs:
    for col in timestamp_cols:
        if col in df.columns:
            df[col] = pd.to_datetime(df[col], utc=True,
errors='coerce')

# 2. Extract and uniform the core columns from each channel
calls_sub = df_calls[['account_id',
'event_at']].rename(columns={'event_at': 'interaction_time'})
calls_sub['channel'] = 'call'

wa_sub = df_whatsapp_events[['account_id',
'event_at']].rename(columns={'event_at': 'interaction_time'})
wa_sub['channel'] = 'whatsapp'

sms_sub = df_sms_events[['account_id',
'event_at']].rename(columns={'event_at': 'interaction_time'})
sms_sub['channel'] = 'sms'

field_sub = df_field_visits[['account_id',
'event_at']].rename(columns={'event_at': 'interaction_time'})
field_sub['channel'] = 'field_visit'

# 3. Concatenate into the chronological touchpoint ledger
```

```
df_interactions = pd.concat([calls_sub, wa_sub, sms_sub,
                             field_sub]).sort_values(['account_id', 'interaction_time'])
```

The 72-Hour Attribution Merge

Map valid payments to the closest preceding interaction and enforce the 3-day operational window to eliminate organic self-cures.

Attribution Logic

Python

```
# 1. Sort both dataframes chronologically for the asof merge
df_clean_payments = df_clean_payments.sort_values('event_at')
df_interactions = df_interactions.sort_values('interaction_time')

# 2. Backward merge: Match payment to the most recent prior
interaction
df_attributed = pd.merge_asof(
    df_clean_payments,
    df_interactions,
    left_on='event_at',
    right_on='interaction_time',
    by='account_id',
    direction='backward'
)

# 3. Filter out ghost accounts from the attributed revenue
df_attributed =
df_attributed[df_attributed['account_id'].isin(df_accounts_clean['account_id'])]

# 4. Calculate attribution window and enforce the 72-hour
operational cutoff
df_attributed['time_to_pay_hours'] = (df_attributed['event_at'] -
df_attributed['interaction_time']).dt.total_seconds() / 3600
df_true_attribution =
df_attributed[df_attributed['time_to_pay_hours'] <= 72]
```

Vendor Disposition Cleansing

Fix the redundant definitional bleed and exclude the corrupt August cohort from vendor telephony metrics.

Disposition Cleaning and Export

Python

```
# 1. Consolidate fragmented success codes
df_call_dispositions['disposition_code'] =
df_call_dispositions['disposition_code'].replace({'PTP':
'PROMISE_TO_PAY'})

# 2. Extract month and permanently drop the incomplete August 2026
data
df_call_dispositions['month'] =
df_call_dispositions['event_at'].dt.to_period('M')
df_call_dispositions =
df_call_dispositions[df_call_dispositions['month'] < '2026-08']

# 3. Export the finalized Golden Datasets
df_master_entity.to_csv('golden_master_entity.csv', index=False)
df_true_attribution.to_csv('golden_attribution_ledger.csv',
index=False)
df_call_dispositions.to_csv('golden_vendor_dispositions.csv',
index=False)
```

Are you planning to load these finalized CSVs directly into a BI tool like Tableau for the presentation, or will you build the charts using [matplotlib/seaborn](#) in Python?